Lab 2: Data Versioning and Validation

DVC Tracking and Great Expectations Data Validation

Submitted by	Mohd Faizy
Date	31 October 2026
Tools	Git, DVC, Great Expectations (1.x), pandas, Python 3.13, VS Code (PowerShell)
Project folder	mlops-lab\lab2

1. Objective

This lab has two parts. **Task 1** uses Git and DVC to version a raw CSV dataset: track it, change it by adding 5 rows, track the new version, and restore the original with `dvc checkout`. **Task 2** builds a data validation check with Great Expectations: an Expectation Suite that verifies one column has no null values and that a numeric column stays within 0 to 100.

2. Task 1: DVC Tracking

2.1 Initialize Git and DVC

A Git repository and a DVC repository were initialized in the project folder and committed as the first commit (`6658658 Initialize Git and DVC` in the log below).

```
git init
dvc init
git commit -m "Initialize Git and DVC"
```

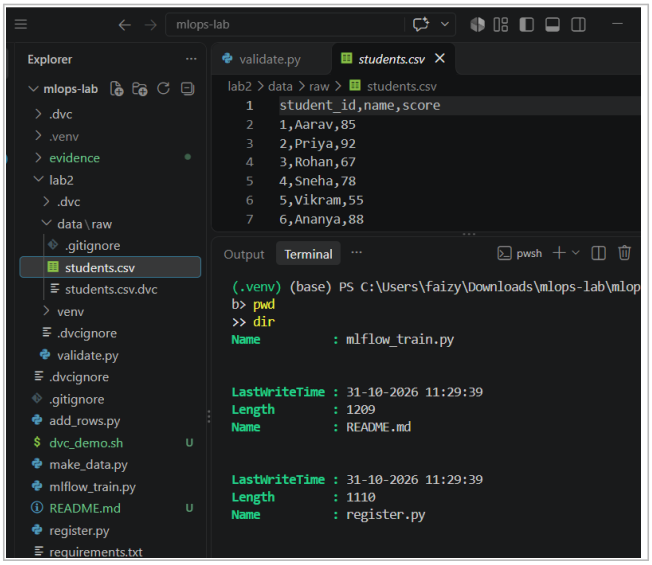


Figure 1: Project structure. lab2 contains .dvc, data\raw\students.csv, students.csv.dvc (pointer file), the data\raw\.gitignore created by DVC, and validate.py.

2.2 Create the raw dataset and track version 1

The raw dataset `data/raw/students.csv` has the columns `student_id, name, score` and 5 rows (student_id 1 to 5).

```
student_id,name,score
1,Aarav,85
2,Priya,92
3,Rohan,67
4,Sneha,78
5,Vikram,55
```

It was then tracked with DVC. `dvc add` moves the data into DVC's cache, creates the small pointer file `students.csv.dvc`, and adds the CSV to `.gitignore`, so Git versions only the pointer file.

```
dvc add data/raw/students.csv
git add data/raw/students.csv.dvc data/raw/.gitignore
git commit -m "Track raw dataset v1 (5 rows)"
git tag v1
```

2.3 Add 5 new rows and track version 2

Five rows (student_id 6 to 10) were appended. The Bash here-document (`cat >> file << 'EOF'`) is not valid in PowerShell and gave a ParserError, so the same rows were appended with a PowerShell here-string piped into `Add-Content`.

```
@'
6,Ananya,88
7,Karan,73
8,Meera,95
9,Arjun,61
10,Diya,82
'@ | Add-Content -Path data/raw/students.csv

dvc add data/raw/students.csv
git add data/raw/students.csv.dvc
git commit -m "Track raw dataset v2 (10 rows)"
git tag v2
```

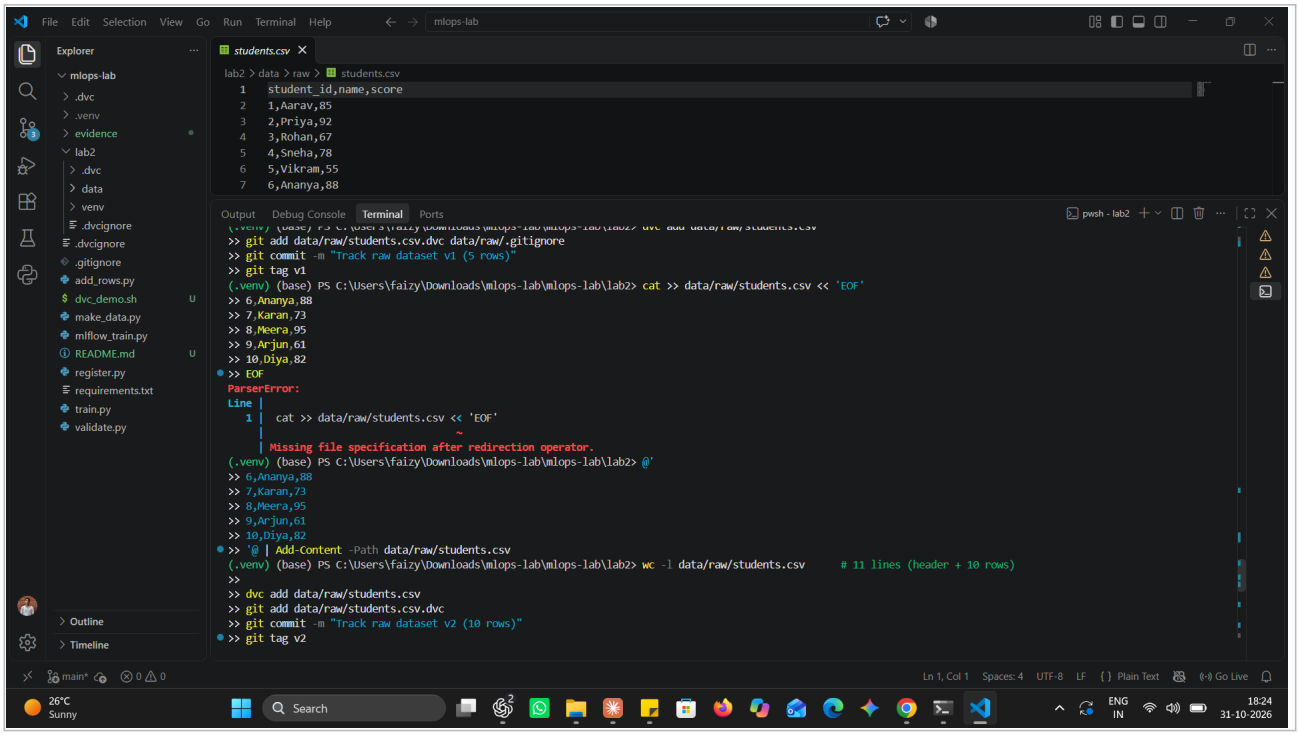


Figure 2 (full screenshot): Version 1 committed and tagged, the failed Bash here-document, the PowerShell here-string that appended rows 6 to 10, and version 2 committed and tagged.

2.4 Revert to the original with dvc checkout

DVC restores the data version referenced by the `.dvc` pointer file. So the pointer file was first moved back to its v1 state with Git, and then `dvc checkout` restored the matching data.

```
git checkout v1 -- data/raw/students.csv.dvc
dvc checkout
cat data/raw/students.csv
```

Result: the file was restored to the original version. It contains the header and the original 5 rows (student_id 1 to 5), and rows 6 to 10 are gone. DVC reported `M data\raw\students.csv` (file modified, i.e. replaced with the cached v1 content).

To confirm that both versions are recoverable, version 2 was then restored the same way:

```
git checkout v2 -- data/raw/students.csv.dvc
dvc checkout
```

Note: `wc -l` is a Linux/Bash command and is not recognized in PowerShell, so it printed an error. The equivalent PowerShell command is `(Get-Content data\raw\students.csv).Count`. The restored contents shown by `cat` already confirm the file has 5 data rows.

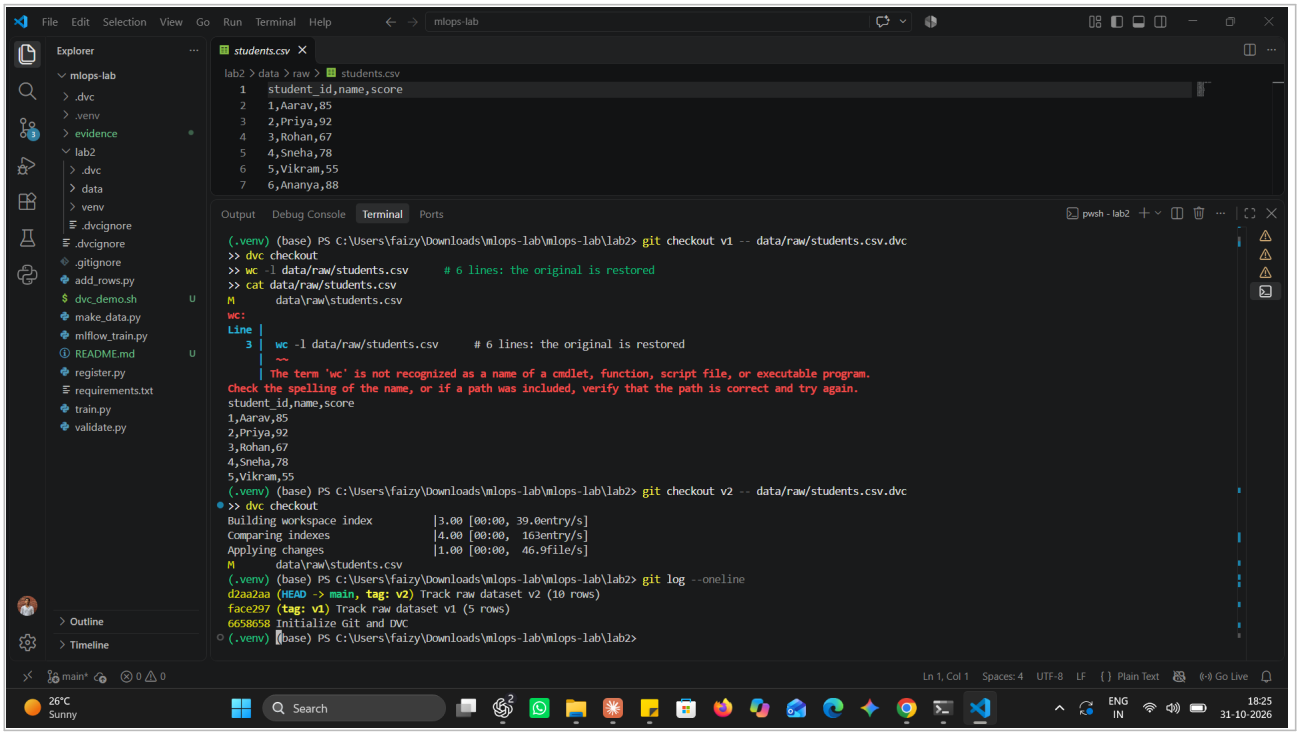


Figure 3 (full screenshot): git checkout v1 followed by dvc checkout restores the 5-row file; restoring v2 afterwards; and git log --oneline showing the commits and tags.

2.5 Git history

`git log --oneline` shows the three commits and the two version tags:

Commit	Tag	Message	Data
6658658	-	Initialize Git and DVC	No data yet
face297	v1	Track raw dataset v1 (5 rows)	5 rows
d2aa2aa	v2 (HEAD -> main)	Track raw dataset v2 (10 rows)	10 rows

Task 1 outcome: the dataset was tracked with DVC, modified with 5 new rows, tracked again as a new version, and reverted to its original state using `dvc checkout`.

3. Task 2: Data Validation with Great Expectations

3.1 Approach

The script `validate.py` loads the CSV with pandas and connects it to Great Expectations through a data source, a dataframe asset, and a batch definition. It creates an Expectation Suite named `students_suite` with two expectations, then validates the batch against the suite.

- **Expectation 1:** `expect_column_values_to_not_be_null` on `student_id` (the column has no null values).
- **Expectation 2:** `expect_column_values_to_be_between` on `score` with `min_value=0` and `max_value=100`.

3.2 Code (validate.py)

```
import great_expectations as gx
import great_expectations.expectations as gxe
import pandas as pd

# 1. Create a context and load the data
context = gx.get_context()
df = pd.read_csv("data/raw/students.csv")

# 2. Connect the dataset (data source -> asset -> batch definition)
data_source = context.data_sources.add_pandas("students_source")
asset = data_source.add_dataframe_asset("students_asset")
batch_def = asset.add_batch_definition_whole_dataframe("students_batch")
batch = batch_def.get_batch(batch_parameters={"dataframe": df})

# 3. Create the Expectation Suite
suite = context.suites.add(gx.ExpectationSuite(name="students_suite"))

# Expectation 1: student_id has no nulls
suite.add_expectation(
    gxe.ExpectColumnValuesToNotBeNull(column="student_id")
)

# Expectation 2: score is between 0 and 100
suite.add_expectation(
    gxe.ExpectColumnValuesToBeBetween(column="score", min_value=0, max_value=100)
)

# 4. Execute the validation
result = batch.validate(suite)
print("Overall success:", result.success)
for r in result.results:
    print(r.expectation_config.type, "->", r.success)
```

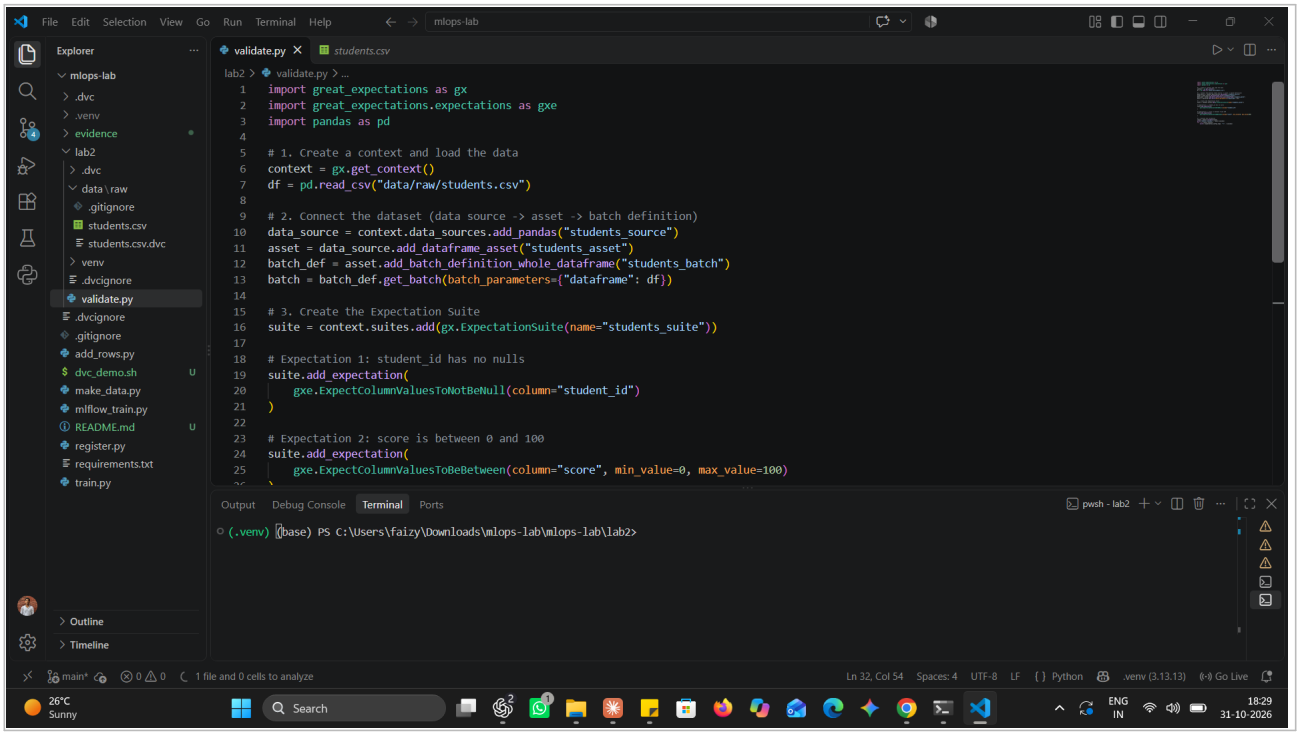


Figure 4 (full screenshot): validate.py in VS Code with the Expectation Suite code.

3.3 Execution and results

The script was run from inside the `lab2` folder on the 10-row version 2 dataset:

```
cd lab2
python validate.py
```

```
Calculating Metrics: 100%|##########| 15/15
Overall success: True
expect_column_values_to_not_be_null -> True
expect_column_values_to_be_between -> True
```

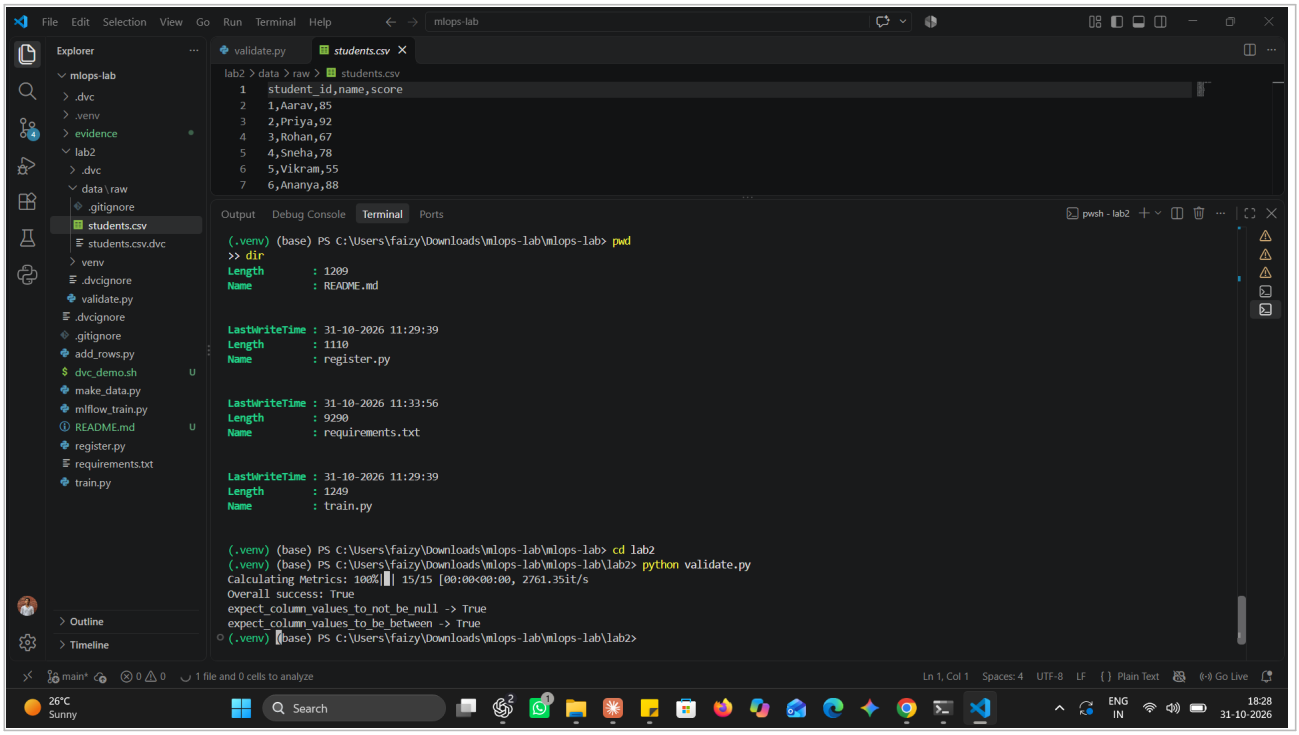


Figure 5 (full screenshot): Validation run. Overall success is True and both expectations pass.

Expectation	Column	Condition	Result
expect_column_values_to_not_be_null	student_id	No null values	PASS
expect_column_values_to_be_between	score	0 to 100	PASS

Interpretation: every `student_id` is populated, and every score (range 55 to 95) lies within 0 to 100, so the whole suite passed.

Note on bounds: by default `ExpectColumnValuesToBeBetween` treats the limits as inclusive. For strictly exclusive limits, add `strict_min=True, strict_max=True`. Since the lowest score is 55 and the highest is 95, the data also satisfies the strict version.

4. Conclusion

Git and DVC together gave reproducible versions of the dataset: Git stores the lightweight pointer files, DVC stores the data, and `git checkout` plus `dvc checkout` moved the CSV between its 5-row and 10-row versions. Great Expectations then added an automated quality gate: the Expectation Suite confirmed that `student_id` has no nulls and that `score` stays between 0 and 100. Together, data versioning and validation make an ML pipeline easier to reproduce and to trust.